

TECH CHALLENGE – FASE 3

FIAP + alura

POSTECH

DEVOPS &
ARQUITETURA CLOUD

Turma: 1DCLT

Integrantes:

Wellington Kitahara – RM367487

Vinicius Cantilio – RM367160

Juliano Lopes – RM368967

MAR, 2026

Sumário

1. INTRODUÇÃO E EVOLUÇÃO DO PROJETO.....	2
1.1. Resumo.....	2
1.2. Contextualização Técnica e Justificativa	3
1.3. Infraestrutura Imutável e Auditável (IaC).....	3
1.4. Segurança Shift-Left (DevSecOps).....	3
1.5. Controle de Deploy com GitOps	3
2. A Solução Proposta: Arquitetura ToggleMaster	4
2.1. IaC Completa	4
2.2. Pipeline DevSecOps Integrado.....	4
2.3. Entrega Contínua (GitOps).....	4
3. Desafio da Fase 3	5
3.1. ToggleMaster: Automação Integral e Governança GitOps.....	5
3.2. Infraestrutura como Código (Terraform).....	5
3.3. Pipeline DevSecOps (CI).....	5
3.4. Entrega Contínua (GitOps com ArgoCD).....	6
4. EVOLUÇÃO ENTRE FASES.....	7
4.1. Detalhamento do Ecossistema de Microsserviços	8
4.2. Visão Estruturada dos Microsserviços	8
4.3. Automação na Fase 3.....	9
Infraestrutura Imutável (Terraform)	9
Segurança Integrada (DevSecOps)	9
Entrega Contínua (GitOps)	9
5. Infraestrutura, Automação e Escalabilidade (IaC, GitOps e HPA)	10
5.1. Terraform (Infrastructure as Code)	10
5.2. Amazon EKS (Orquestração e Resiliência)	10
5.3. DevSecOps (CI com Segurança Integrada).....	11
5.4. GitOps com ArgoCD (Entrega Contínua).....	11
5.5. Application Load Balancer (ALB) / Ingress Controller	11
5.6. Horizontal Pod Autoscaler (HPA).....	12
6. DIFERENÇAS DE ARQUITETURAS FASES 1 A 3.....	13
6.1. Fase 1 — Arquitetura Monolítica (MVP)	13
6.2. Características Principais	13
6.3. Limitações Arquiteturais	13
6.4. Resumo da Fase 1	14
6.5. Fase 2 — Arquitetura Distribuída (Microsserviços).....	16
6.6. Evoluções em Relação à Fase 1	16
6.7. Componentes Arquiteturais	16
Orquestração.....	16
Acesso e Roteamento	16

Persistência	16
Escalabilidade	17
6.8. Fluxos de Comunicação	17
Síncrono	17
Assíncrono	17
6.9. Limitações da Fase 2.....	17
6.10. Resumo da Fase 2	17
6.11. Fase 3 — Arquitetura Automatizada (IaC, DevSecOps e GitOps)	19
6.12. Evoluções em Relação à Fase 2	19
6.13. Camadas da Arquitetura	19
Infraestrutura (IaC — Terraform)	19
Pipeline DevSecOps (CI).....	19
GitOps (Entrega Contínua com ArgoCD).....	20
Fluxo Operacional (Nova Topologia Lógica)	20
6.14. Escalabilidade e Resiliência	20
6.15. Ganhos em Relação à Fase 2	20
6.16. Resumo da Fase 3	21
6.17. Detalhamento do Fluxo de Automação (CI, IaC e GitOps).....	21
7. ESTRATÉGIA DE VALIDAÇÃO E TESTES DE INFRAESTRUTURA	24
7.1. Validação da Automação (IaC).....	24
7.2. Validação do Pipeline DevSecOps	24
7.3. Validação do GitOps.....	24
7.4. Validação do Deploy Automatizado	24
7.5. (Opcional) Comportamento do cluster	25
Validação da Automação (IaC).....	26
Validação do Pipeline DevSecOps	28
Validação do GitOps	29
Validação do Deploy Automatizado.....	31
(Opcional) Comportamento do cluster	32
8. CONCLUSÃO E ANÁLISE	33
8.1. Vantagens da Automação e Governança (IaC, DevSecOps e GitOps).....	33
Infraestrutura Imutável e Reprodutível	33
Segurança Integrada (Shift-Left).....	33
Consistência do Deploy (GitOps).....	33
Escalabilidade Mantida	34
8.2. Complexidades Remanescentes e Decisões Técnicas.....	34
Complexidade do Setup Inicial.....	34
Limitação de Escalabilidade	34
8.3. Considerações Finais	34
9. 12-Factor App e AWS Well-Architected Framework	36
9.1. Adoção de Princípios Modernos de Arquitetura.....	36

9.2. Aplicação do Modelo Twelve-Factor App.....	36
9.3. Codebase e Versionamento	36
9.4. Configuração Externa (Config)	37
9.5. Backing Services como Recursos Plugáveis	37
9.6. Build, Release e Run (Separação de Etapas).....	37
9.7. Processos Stateless e Escalabilidade	37
9.8. Disposability e Resiliência	38
9.9. Paridade entre Ambientes (Dev/Prod Parity)	38
9.10. Resumo do Twelve-Factor no Projeto.....	38
10. Aplicação do AWS Well-Architected Framework	39
10.1. Excelência Operacional	39
10.2. Segurança	39
10.3. Confiabilidade (Reliability)	39
10.4. Eficiência de Performance	40
10.5. Otimização de Custos.....	40
10.6. Integração entre os Frameworks	40
11. Custos.....	42
LINKS E DOCUMENTAÇÕES	43
REFERÊNCIAS BIBLIOGRÁFICAS	44

1. INTRODUÇÃO E EVOLUÇÃO DO PROJETO

O projeto ToggleMaster, que na Fase 2 consolidou a arquitetura de Microsserviços e a orquestração via Amazon EKS (Kubernetes), entra agora na Fase 3 focado na sustentabilidade operacional e segurança.

Esta fase é guiada pelo princípio fundamental de Infraestrutura Imutável e pelo mandato:

"Se não está no código, não existe".

O objetivo é eliminar a complexidade operacional introduzida pela Fase 2 (como conflitos de deploy e credenciais inseguras) através da completa automação.

A evolução é marcada pela adoção de Infraestrutura como Código (IaC) com Terraform, pipelines de segurança DevSecOps, e a implementação do padrão GitOps utilizando ArgoCD para a Entrega Contínua.

Esta abordagem garante um ambiente auditável e que incorpora a segurança desde o início do ciclo de vida do software, alinhando-se aos princípios apresentados nas aulas de DevOps e Arquitetura Cloud da FIAP, que enfatizam a automação e a segurança como pilares fundamentais da maturidade operacional, bem como às práticas descritas por Humble e Farley (Continuous Delivery, 2010).

1.1. Resumo

Este trabalho detalha a transição do ToggleMaster para um modelo de operações DevSecOps e GitOps.

A solução endereça a complexidade operacional da arquitetura distribuída (Fase 2) através da automação integral do ambiente com Terraform, garantindo a criação reprodutível de todos os recursos AWS (VPC, EKS, RDS, ElastiCache, DynamoDB e SQS).

O ciclo de vida dos cinco microsserviços é agora regido por workflows de Integração Contínua (CI) que incorporam estágios obrigatórios de Security Scan (SAST/SCA) e Container Scan.

O pipeline é configurado para falhar e bloquear o processo se uma vulnerabilidade CRÍTICA for detectada.

A entrega final é efetuada pelo padrão GitOps, onde o ArgoCD monitora o repositório de manifestos Kubernetes e sincroniza automaticamente o estado desejado no cluster EKS, seguindo o modelo declarativo amplamente difundido pela Cloud Native Computing Foundation (CNCF, 2022)

1.2. Contextualização Técnica e Justificativa

Após a adoção da arquitetura de microsserviços, a prioridade se deslocou para a governança da mudança e a segurança na cadeia de suprimentos de software.

Lançamentos manuais (via `kubectl apply`) e a falta de controle de infraestrutura estavam gerando conflitos de versão e falhas de segurança, como vulnerabilidades em bibliotecas que chegaram à produção.

A Fase 3 justifica-se pela necessidade de eliminar esses riscos operacionais e garantir a conformidade, conforme abordado no conceito de DevSecOps e segurança Shift-Left apresentado nas aulas da FIAP e nas diretrizes da OWASP (2021).

1.3. Infraestrutura Imutável e Auditável (IaC)

O uso do Terraform garante que os ambientes possam ser recriados de forma idêntica e em tempo reduzido (contrastando com a recriação manual que levava dias), promovendo a previsibilidade e a rastreabilidade total das mudanças, conforme discutido por Kief Morris (Infrastructure as Code, 2016).

1.4. Segurança Shift-Left (DevSecOps)

A inserção de Security Scans (SAST e SCA) diretamente no pipeline de CI move a responsabilidade da segurança para o início do ciclo de desenvolvimento, prevenindo proativamente a entrada de código vulnerável em produção, em linha com as recomendações do OWASP Top 10 (2021) e práticas modernas de DevSecOps.

1.5. Controle de Deploy com GitOps

A adoção do ArgoCD e do padrão GitOps resolve o problema de "drift" e os conflitos de versão gerados pelo `kubectl apply` manual dos desenvolvedores.

O estado do cluster EKS passa a ser ditado unicamente pelo repositório Git, seguindo o modelo proposto por Weaveworks (GitOps, 2020) e amplamente adotado no ecossistema cloud-native.

2. A Solução Proposta: Arquitetura ToggleMaster

A plataforma, em sua Fase 3, foca na automatização total e no controle do pipeline de entrega:

2.1. IaC Completa

O provisionamento de toda a infraestrutura AWS é gerenciado via Terraform, com a configuração obrigatória do Backend Remoto S3 para gerenciamento do estado, garantindo consistência e colaboração entre equipes, conforme práticas recomendadas pela HashiCorp.

2.2. Pipeline DevSecOps Integrado

Workflows de CI são implementados para os 5 microsserviços, cobrindo Build & Test, Linter, SAST/SCA e Container Scan, com bloqueio de Critical, reforçando a abordagem de segurança contínua defendida pela OWASP e pelas práticas modernas de engenharia de software seguro.

2.3. Entrega Contínua (GitOps)

O deploy é feito de forma declarativa e automatizada, onde o pipeline de CI atualizar a tag da imagem no repositório de manifestos, e o ArgoCD sincroniza automaticamente a nova versão para o cluster EKS, garantindo consistência entre o estado desejado e o estado real da infraestrutura.

3. Desafio da Fase 3

O desafio central para esta fase do projeto consiste em resolver a insustentabilidade operacional e a insegurança introduzidas pela arquitetura de microsserviços da Fase 2.

Com desenvolvedores executando deploys manuais (via kubectl apply), gerenciamento de credenciais inseguro e ambientes de homologação recriados manualmente (levando dias), a prioridade é impor o conceito de Infraestrutura Imutável sob o mandato:

"Se não está no código, não existe".

O foco é automatizar todo o ciclo de vida do software e da infraestrutura, integrando segurança em cada etapa (DevSecOps) e adotando o GitOps para entrega contínua, conforme abordado nas aulas de DevOps e Cloud da FIAP e nas práticas modernas descritas por Humble e Farley (2010).

3.1. ToggleMaster: Automação Integral e Governança GitOps

A concretização desta arquitetura envolve os seguintes requisitos técnicos:

3.2. Infraestrutura como Código (Terraform)

Provisionamento completo de todos os recursos AWS utilizados na Fase 2:

- VPC
- Cluster EKS (associado à LabRole)
- 3 instâncias RDS (PostgreSQL)
- 1 Cluster ElastiCache (Redis)
- 1 Tabela DynamoDB
- 1 Fila SQS

Configuração obrigatória do Backend Remoto S3 para gerenciar o terraform.tfstate, garantindo consistência e versionamento do estado da infraestrutura, conforme práticas recomendadas pela HashiCorp (Terraform Documentation).

3.3. Pipeline DevSecOps (CI)

Implementação de workflows de CI para os 5 microsserviços com os estágios de:

- Build & Test
- Linter/Static Analysis
- Security Scan (SAST & SCA)

Regra de Bloqueio:

O pipeline deve falhar e bloquear o processo se uma vulnerabilidade CRÍTICA for detectada.

Construção da imagem Docker, execução de Container Scan (Trivy) e push para o ECR com a tag do commit hash, alinhando-se às práticas de segurança contínua recomendadas pela OWASP (2021).

3.4. Entrega Contínua (GitOps com ArgoCD)

Instalação e configuração do ArgoCD no cluster EKS.

Criação de um repositório GitOps exclusivo para manifestos Kubernetes.

O pipeline de CI deve, após o push da imagem, atualizar a tag no manifesto Kubernetes do repositório GitOps.

O ArgoCD deve monitorar este repositório e sincronizar automaticamente o estado desejado para o cluster EKS, seguindo o modelo declarativo promovido pela CNCF e pelo conceito de GitOps (Weaveworks, 2020).

4. EVOLUÇÃO ENTRE FASES

O projeto **ToggleMaster**, concebido na Fase 1 como um MVP monolítico para validar a centralização de *Feature Flags*, evoluiu nesta Fase 2 para uma arquitetura moderna de microsserviços distribuídos. Esta transição visou eliminar gargalos de performance e permitir uma escalabilidade robusta em nuvem. A **Fase 3** complementa esta evolução ao focar na **sustentabilidade operacional** e na **segurança**, aplicando o conceito de **Infraestrutura Imutável**.

Atributo	Fase 1: Monólito (MVP)	Fase 2: Microsserviços (EKS)	Fase 3: Automação (IaC/GitOps/DevSecOps)	Vantagem Técnica
Escalabilidade	Vertical (Resize de EC2)	Horizontal (HPA por Pod)	Infraestrutura EKS provisionada via IaC (Terraform)	Ajuste elástico conforme a demanda e infraestrutura reprodutível.
Disponibilidade	Ponto único de falha	Isolamento de falhas por serviço	Ambientes Imutáveis e Recriáveis (IaC)	Resiliência; eliminação de <i>drift</i> e rápida recuperação.
Persistência	Banco de dados único	Persistência Poliglota (RDS, Redis, Dynamo)	Recursos de dados provisionados via IaC (Terraform)	Uso do banco ideal e provisionamento automatizado de múltiplos <i>data stores</i> .
Deployment	Atualização total da aplicação	<i>Deploy</i> independente por container	Entrega Contínua (GitOps com ArgoCD)	Agilidade, menor risco e <i>sync</i> automático do cluster EKS.
Performance	Limitada por latência de disco	Alta performance com Cache (Redis)	Qualidade garantida por Pipeline CI/CD	Respostas em milissegundos no motor de avaliação.
Governança/Segurança	N/A	Manual/Ad-hoc	DevSecOps (SAST/SCA/Container Scan)	Segurança <i>shift-left</i> : Bloqueio automático de vulnerabilidades CRÍTICAS .
Operação	Manual/Console	Manual/CLI (<i>Kubectl</i> de máquinas locais)	Infraestrutura como Código (Terraform + Backend S3)	Estado da infraestrutura versionado e gestão de <i>state</i> remoto seguro.

4.1. Detalhamento do Ecossistema de Microserviços

A arquitetura monolítica foi objeto de decomposição em um total de cinco (5) serviços especializados, visando a maximização do desacoplamento funcional e a otimização da eficiência operacional.

Com base no detalhamento da Fase 2 e nos requisitos da Fase 3 fornecidos no documento, esta seção apresenta como a automação (IaC, CI/CD e GitOps) transformou a gestão de cada um dos cinco serviços.

Na Fase 3, a arquitetura de microserviços do ToggleMaster é inteiramente gerenciada por automação. A implantação de cada serviço, desde o provisionamento da infraestrutura até o deployment no cluster, segue um fluxo GitOps/DevSecOps que elimina intervenções manuais, conforme abordado nas aulas de DevOps e Arquitetura Cloud da FIAP e nos princípios de Continuous Delivery descritos por Humble e Farley (2010).

4.2. Visão Estruturada dos Microserviços

Microserviço	Linguagem	Persistência Provisionada (Terraform)	Pipeline de CI/CD (Automação)
Auth-Service	Go	AWS RDS (PostgreSQL)	Orquestrado via ArgoCD . Imagem construída com Build & Test e verificada com Security Scan (SAST) .
Flag-Service	Python	AWS RDS (PostgreSQL)	Pipeline CI rodando Linter/Static Analysis e Security Scan (SCA) . A nova tag é atualizada automaticamente no repositório GitOps.
Targeting-Service	Python	AWS RDS (PostgreSQL)	Pipeline com Regra de Bloqueio que falha se uma vulnerabilidade CRÍTICA for encontrada. O <i>deploy</i> é sincronizado pelo ArgoCD.
Evaluation-Service	Go	AWS ElastiCache (Redis) e AWS SQS	O pipeline de CI constrói e realiza Container Scan com Trivy antes do <i>push</i> para o AWS ECR.
Analytics-Service	Python	Amazon DynamoDB (ToggleMasterAnalytics) e AWS SQS	Implementado como um <i>worker</i> assíncrono. Sua infraestrutura (DynamoDB e SQS) é criada via Terraform e o <i>deploy</i> é gerenciado via GitOps.

4.3. Automação na Fase 3

Infraestrutura Imutável (Terraform)

Todos os data stores (RDS, ElastiCache, DynamoDB) e recursos de rede/mensageria (SQS) são provisionados e gerenciados via código Terraform, garantindo reprodutibilidade e rastreabilidade da infraestrutura, conforme práticas recomendadas pela HashiCorp.

Segurança Integrada (DevSecOps)

O código de cada microsserviço é submetido a scans de segurança (SAST e SCA) e suas imagens Docker passam por Container Scan antes de serem enviadas ao ECR, alinhando-se às recomendações de segurança contínua da OWASP (2021).

Entrega Contínua (GitOps)

O deploy dos manifestos Kubernetes de cada um dos cinco serviços é monitorado e executado automaticamente pelo ArgoCD, garantindo que o estado do cluster EKS reflita apenas o que está versionado no repositório GitOps, conforme o modelo declarativo promovido pela CNCF (2022).

5. Infraestrutura, Automação e Escalabilidade (IaC, GitOps e HPA)

Em evolução à abordagem da Fase 2 — que já introduzia a orquestração via Kubernetes — a Fase 3 estabelece um modelo totalmente automatizado e imutável de infraestrutura, baseado nos princípios de **Infrastructure as Code (IaC)**, **DevSecOps** e **GitOps**.

Neste contexto, toda a infraestrutura e o ciclo de vida dos microsserviços passam a ser definidos e versionados como código, eliminando dependências de configurações manuais e garantindo reprodutibilidade, rastreabilidade e governança, conforme abordado nas aulas de DevOps e Arquitetura Cloud da FIAP e nos princípios de engenharia de software modernos descritos por Morris (2016).

5.1. Terraform (Infrastructure as Code)

A provisão de todos os recursos de infraestrutura é realizada por meio do Terraform, com arquitetura modular e backend remoto em S3 para gerenciamento seguro do estado.

São provisionados automaticamente:

- VPC, subnets públicas e privadas, Internet Gateway e NAT Gateway
- Cluster Amazon EKS e Node Groups
- Bancos de dados (Amazon RDS PostgreSQL)
- Cache (Amazon ElastiCache Redis)
- Banco NoSQL (Amazon DynamoDB)
- Mensageria (Amazon SQS)
- Repositórios de contêiner (Amazon ECR)

Essa abordagem garante infraestrutura imutável, versionada e reproduzível em qualquer ambiente, conforme práticas recomendadas pela HashiCorp.

5.2. Amazon EKS (Orquestração e Resiliência)

O Amazon EKS permanece como a plataforma central de orquestração, responsável por:

- Garantir o *Self-Healing* dos pods
- Manter o estado desejado dos deployments
- Executar estratégias de rollout controlado (*rolling updates*)

A integração com o Terraform assegura que o cluster seja totalmente gerenciado como código, alinhando-se aos padrões definidos pela Cloud Native Computing Foundation (CNCF, 2022).

5.3. DevSecOps (CI com Segurança Integrada)

O pipeline de CI/CD foi evoluído para um modelo DevSecOps completo, implementado via GitHub Actions, contemplando:

- Build e testes automatizados
- Análise estática de código (lint + SAST)
- Análise de dependências (SCA)
- Scan de vulnerabilidades em imagens Docker (Trivy)
- Bloqueio automático do pipeline em caso de vulnerabilidades críticas

Além disso, a autenticação com a AWS é realizada via OIDC, eliminando o uso de credenciais estáticas e aumentando significativamente o nível de segurança, conforme boas práticas recomendadas pela OWASP (2021).

5.4. GitOps com ArgoCD (Entrega Contínua)

O processo de deploy foi migrado para um modelo GitOps, onde:

- Os manifestos Kubernetes são armazenados em um repositório dedicado
- O pipeline atualiza automaticamente as tags das imagens
- O ArgoCD monitora o repositório e sincroniza o estado desejado com o cluster

Essa abordagem elimina o uso de comandos manuais (kubectl apply) e garante deploy automatizado, auditável e consistente, conforme o modelo GitOps proposto por Weaveworks (2020).

5.5. Application Load Balancer (ALB) / Ingress Controller

O acesso externo aos serviços é realizado via Ingress Controller (NGINX), integrado ao provisionamento de Load Balancers na AWS, garantindo:

- Roteamento baseado em path (ex: /auth, /flags, etc.)
- Balanceamento de carga entre os pods
- Alta disponibilidade e escalabilidade de entrada

5.6. Horizontal Pod Autoscaler (HPA)

A escalabilidade horizontal continua sendo implementada via HPA, com base em métricas de CPU fornecidas pelo Metrics Server:

- Aplicado aos serviços *evaluation* e *analytics*
- Escalonamento automático quando a utilização média de CPU atinge ~70%
- Ajuste dinâmico do número de réplicas conforme a demanda

Essa estratégia assegura elasticidade e estabilidade operacional, mesmo sob variações significativas de carga, conforme práticas recomendadas pela Kubernetes Documentation.

6. DIFERENÇAS DE ARQUITETURAIS FASES 1 A 3

Abaixo uma breve descrição do projeto desde o princípio atentando-se às nuances de cada fase.

6.1. Fase 1 — Arquitetura Monolítica (MVP)

A Fase 1 do projeto ToggleMaster foi concebida com foco em simplicidade e validação funcional, adotando uma arquitetura monolítica hospedada em uma única instância EC2 dentro de uma VPC na AWS.

Nesta abordagem, toda a lógica da aplicação — incluindo autenticação, gerenciamento de flags e processamento — encontra-se centralizada em um único deploy, compartilhando o mesmo ambiente de execução e o mesmo banco de dados relacional.

6.2. Características Principais

- Aplicação monolítica executando em instância única (EC2)
 - Banco de dados PostgreSQL em subnets privadas (RDS)
 - Acesso externo via Internet (HTTP/HTTPS)
 - Uso de AWS Secrets Manager para gerenciamento de credenciais
 - Comunicação direta entre aplicação e banco
-

6.3. Limitações Arquiteturais

- Ponto único de falha (SPOF)
 - Ausência de escalabilidade horizontal
 - Baixo desacoplamento entre funcionalidades
 - Implantação manual e centralizada
 - Baixa observabilidade e governança
-

6.4. Resumo da Fase 1

A arquitetura monolítica foi adequada para um MVP, permitindo rápida validação do produto. No entanto, suas limitações estruturais tornaram evidente a necessidade de evolução para um modelo distribuído e escalável.

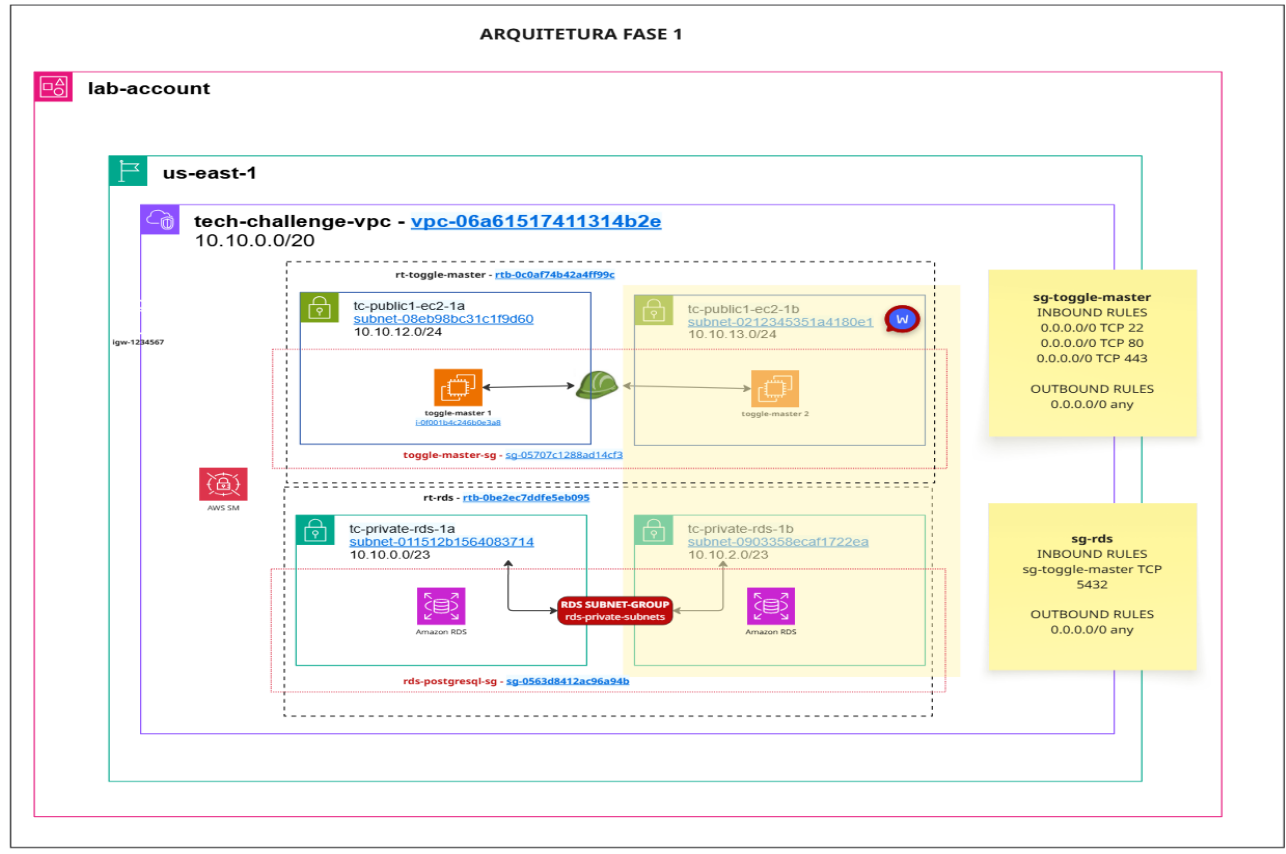


Imagem 1 - Topologia Fase 1 - Arquitetura Monolítica

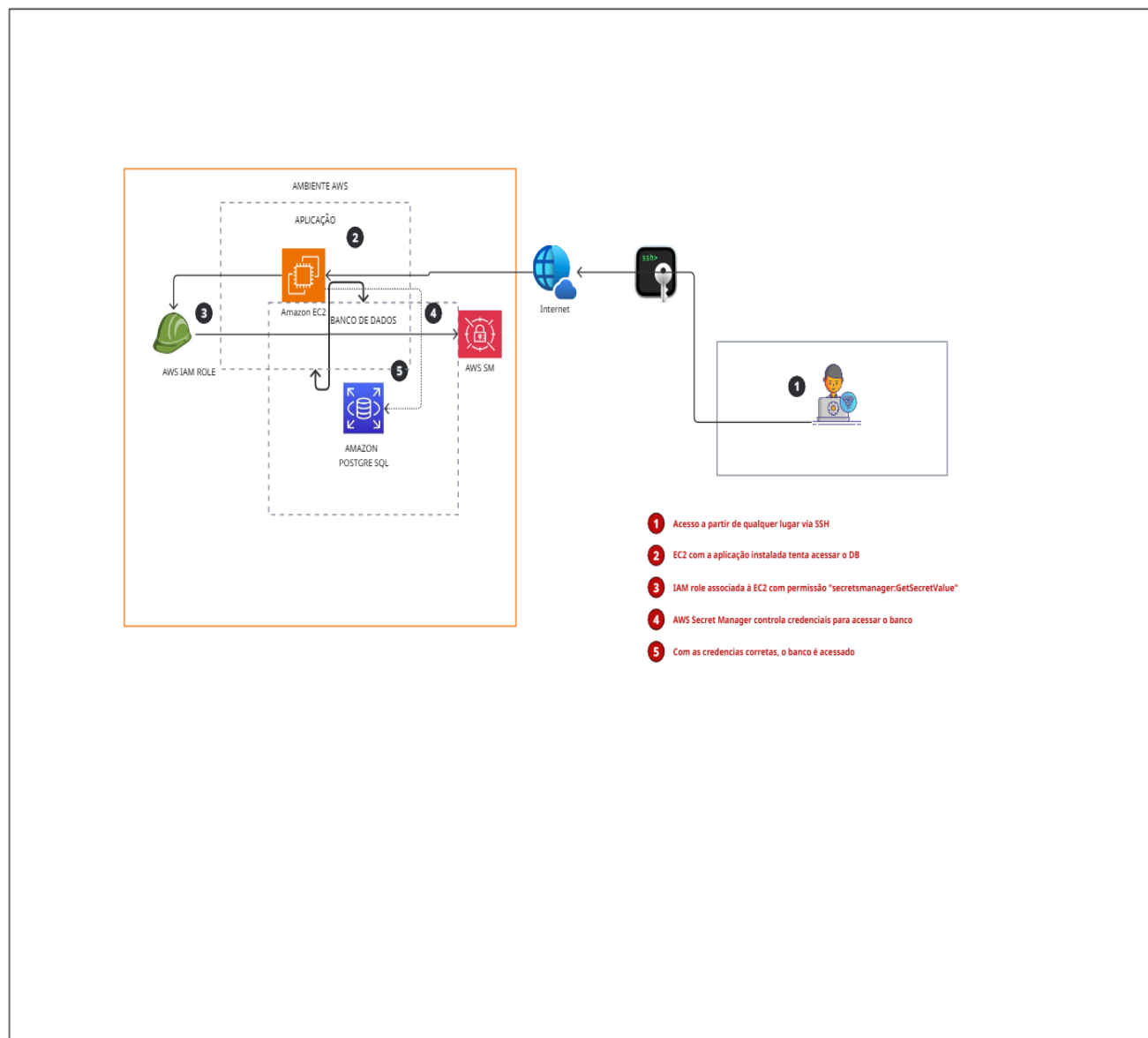


Imagem 2 - Fluxo de Comunicação Fase 1

6.5. Fase 2 — Arquitetura Distribuída (Microserviços)

A Fase 2 representa a evolução para uma arquitetura baseada em microserviços, com foco em escalabilidade, desacoplamento e resiliência.

A aplicação foi decomposta em cinco serviços independentes (auth, flag, targeting, evaluation e analytics), executados em um cluster Kubernetes (Amazon EKS), permitindo isolamento funcional e maior flexibilidade operacional.

6.6. Evoluções em Relação à Fase 1

- Decomposição do monolito em microserviços especializados
 - Introdução do Amazon EKS para orquestração de contêineres
 - Separação de responsabilidades por serviço
 - Introdução de comunicação síncrona e assíncrona
 - Uso de múltiplos data stores (RDS, Redis, DynamoDB, SQS)
-

6.7. Componentes Arquiteturais

Orquestração

- Amazon EKS (Kubernetes)
- Node Groups distribuídos em múltiplas subnets privadas
- Namespaces para organização dos serviços

Acesso e Roteamento

- Application Load Balancer (ALB)
- NGINX Ingress Controller
- Roteamento baseado em path (/auth, /flag, etc.)

Persistência

- RDS (PostgreSQL) para dados transacionais
- Redis (ElastiCache) para cache
- DynamoDB para analytics
- SQS para processamento assíncrono

Escalabilidade

- Horizontal Pod Autoscaler (HPA) baseado em CPU
 - Alta disponibilidade com múltiplas AZs
-

6.8. Fluxos de Comunicação

Síncrono

- Cliente → ALB → Ingress → Microserviços → Banco
- Baixa latência, usado no caminho crítico

Assíncrono

- Serviços → SQS → Worker (analytics) → DynamoDB
 - Processamento desacoplado e resiliente
-

6.9. Limitações da Fase 2

Apesar dos avanços arquiteturais, a Fase 2 ainda apresentava desafios operacionais relevantes:

- Deploy manual via kubectl apply
 - Risco de conflitos de versão (drift)
 - Infraestrutura criada parcialmente de forma manual
 - Ausência de pipelines estruturados (CI/CD)
 - Segurança não integrada ao ciclo de desenvolvimento
-

6.10. Resumo da Fase 2

A arquitetura distribuída trouxe ganhos significativos de escalabilidade e resiliência, porém introduziu complexidade operacional, evidenciando a necessidade de automação, padronização e governança.

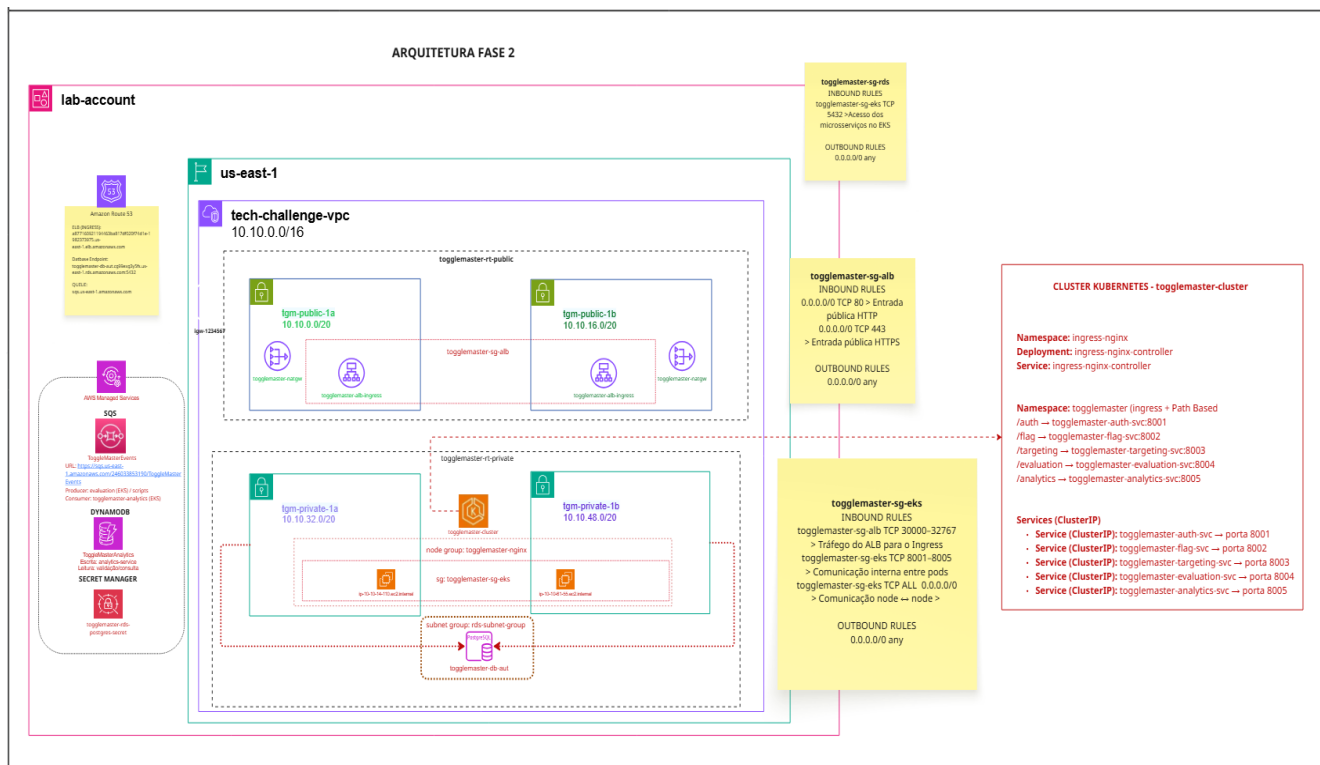


Imagem 3 - Topologia Fase 2 - Arquitetura Microsserviços

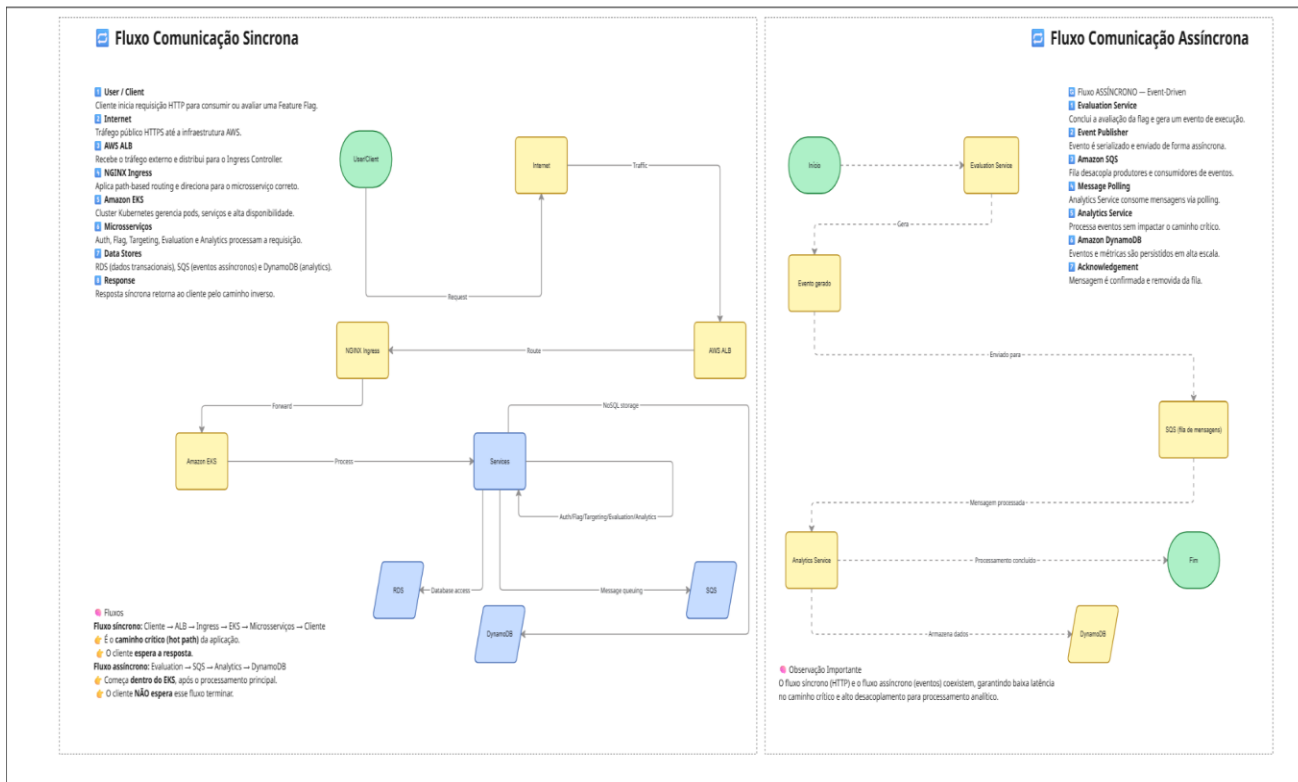


Imagem 4 - Fluxo de Comunicação Fase 2

6.11. Fase 3 — Arquitetura Automatizada (IaC, DevSecOps e GitOps)

A Fase 3 representa a maturidade operacional da solução, transformando a arquitetura distribuída da Fase 2 em um ecossistema totalmente automatizado, seguro e auditável.

A principal diretriz passa a ser o conceito de **Infraestrutura Imutável**, sob o princípio:

"Se não está no código, não existe".

6.12. Evoluções em Relação à Fase 2

- Eliminação completa de processos manuais de deploy
 - Infraestrutura totalmente provisionada via Terraform (IaC)
 - Introdução de pipelines DevSecOps com segurança integrada
 - Adoção do modelo GitOps com ArgoCD
 - Uso de autenticação segura via OIDC (sem credenciais estáticas)
-

6.13. Camadas da Arquitetura

Infraestrutura (IaC — Terraform)

Toda a infraestrutura é definida e versionada como código:

- VPC, subnets, NAT Gateway e Internet Gateway
- Cluster EKS e Node Groups
- RDS, Redis, DynamoDB e SQS
- Repositórios ECR
- Backend remoto S3 para estado do Terraform

🔗 Resultado: **infraestrutura reproduzível, auditável e imutável**

Pipeline DevSecOps (CI)

O ciclo de build e validação passa a ser automatizado:

- Build e testes automatizados

- Linter e análise estática (SAST)
- Análise de dependências (SCA)
- Container Scan (Trivy)
- Bloqueio do pipeline em vulnerabilidades críticas

👉 Resultado: **segurança Shift-Left e prevenção de falhas em produção**

GitOps (Entrega Contínua com ArgoCD)

O modelo de deploy é completamente reformulado:

- Manifestos Kubernetes armazenados em repositório GitOps
- Pipeline atualiza automaticamente as imagens
- ArgoCD sincroniza o estado desejado com o cluster

👉 Resultado:

- Eliminação do uso de kubectl manual
 - Deploy consistente e auditável
 - Controle total via versionamento
-

Fluxo Operacional (Nova Topologia Lógica)

Developer → GitHub → CI Pipeline → ECR → GitOps Repo → ArgoCD → EKS

6.14. Escalabilidade e Resiliência

- HPA mantido (escala em ~70% CPU)
 - Auto-healing garantido pelo Kubernetes
 - Processamento assíncrono desacoplado via SQS
-

6.15. Ganhos em Relação à Fase 2

- Eliminação de drift e conflitos de deploy
- Automação completa do ciclo de vida
- Segurança integrada ao pipeline
- Redução drástica de erros humanos

- Ambientes reproduzíveis em minutos

6.16. Resumo da Fase 3

A Fase 3 consolida a arquitetura do ToggleMaster como uma solução moderna de engenharia de software, combinando escalabilidade, segurança e automação.

A complexidade introduzida pelos microsserviços passa a ser controlada por práticas maduras de DevOps, resultando em um sistema robusto, confiável e preparado para ambientes produtivos.

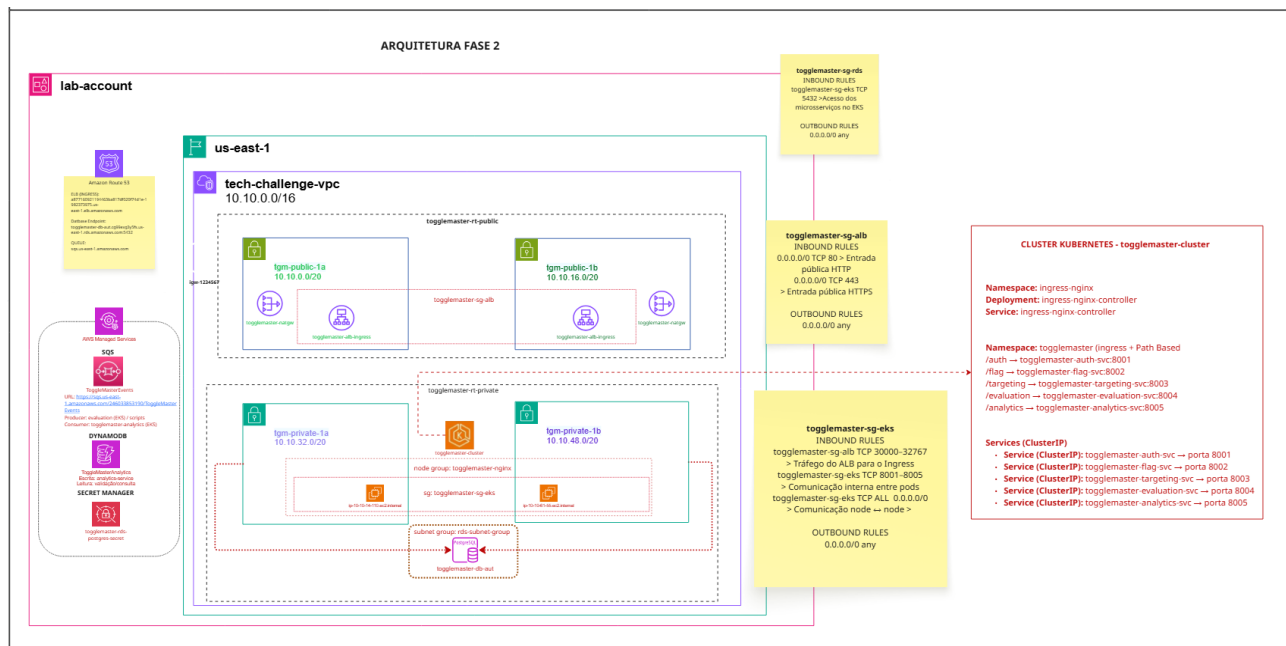


Imagem 5 - Topologia Fase 3 - Arquitetura Microsserviços (Inalterada "Fisicamente")

6.17. Detalhamento do Fluxo de Automação (CI, IaC e GitOps)

A Fase 3 introduz um fluxo operacional totalmente automatizado que integra desenvolvimento, provisionamento de infraestrutura e entrega contínua, garantindo consistência e eliminando intervenções manuais.

Pipeline DevSecOps (CI)

O fluxo é iniciado a partir de alterações realizadas pelo desenvolvedor, que efetua um commit/push no repositório GitHub.

Esse evento aciona automaticamente o pipeline de CI (GitHub Actions), responsável por:

Build e testes automatizados

Lint e análise estática (SAST)

Análise de dependências (SCA)

Container Scan com Trivy

Após a validação, a imagem Docker é construída, versionada e enviada ao Amazon ECR.

🔗 Resultado: garantia de qualidade e segurança antes da disponibilização da aplicação

Atualização do Repositório GitOps

Após o build da imagem, o pipeline atualiza automaticamente os manifestos Kubernetes em um repositório GitOps, substituindo a versão da imagem pela nova tag gerada.

🔗 Resultado: o deploy deixa de ser imperativo e passa a ser declarativo

Reconciliação com ArgoCD (GitOps)

O ArgoCD, executando dentro do cluster EKS, monitora continuamente o repositório GitOps.

Ao detectar alterações:

Sincroniza o estado desejado

Aplica as mudanças no cluster

Garante consistência entre Git e ambiente real

🔗 Resultado: eliminação de drift e deploy automatizado e auditável

Execução no Cluster Kubernetes (EKS)

Após a reconciliação, o Kubernetes executa a estratégia de deployment configurada:

Rolling Update dos pods

Substituição gradual das instâncias antigas

Garantia de alta disponibilidade durante o processo

🔗 Resultado: atualização contínua sem impacto para o usuário final

Provisionamento de Infraestrutura (IaC — Terraform)

De forma complementar ao fluxo de deploy, toda a infraestrutura necessária é previamente provisionada via Terraform:

Criação e configuração da VPC e subnets

Provisionamento do cluster EKS

Configuração dos serviços de dados e mensageria

Gerenciamento do estado via backend remoto (S3)

🔗 Resultado: ambiente totalmente reproduzível, versionado e imutável

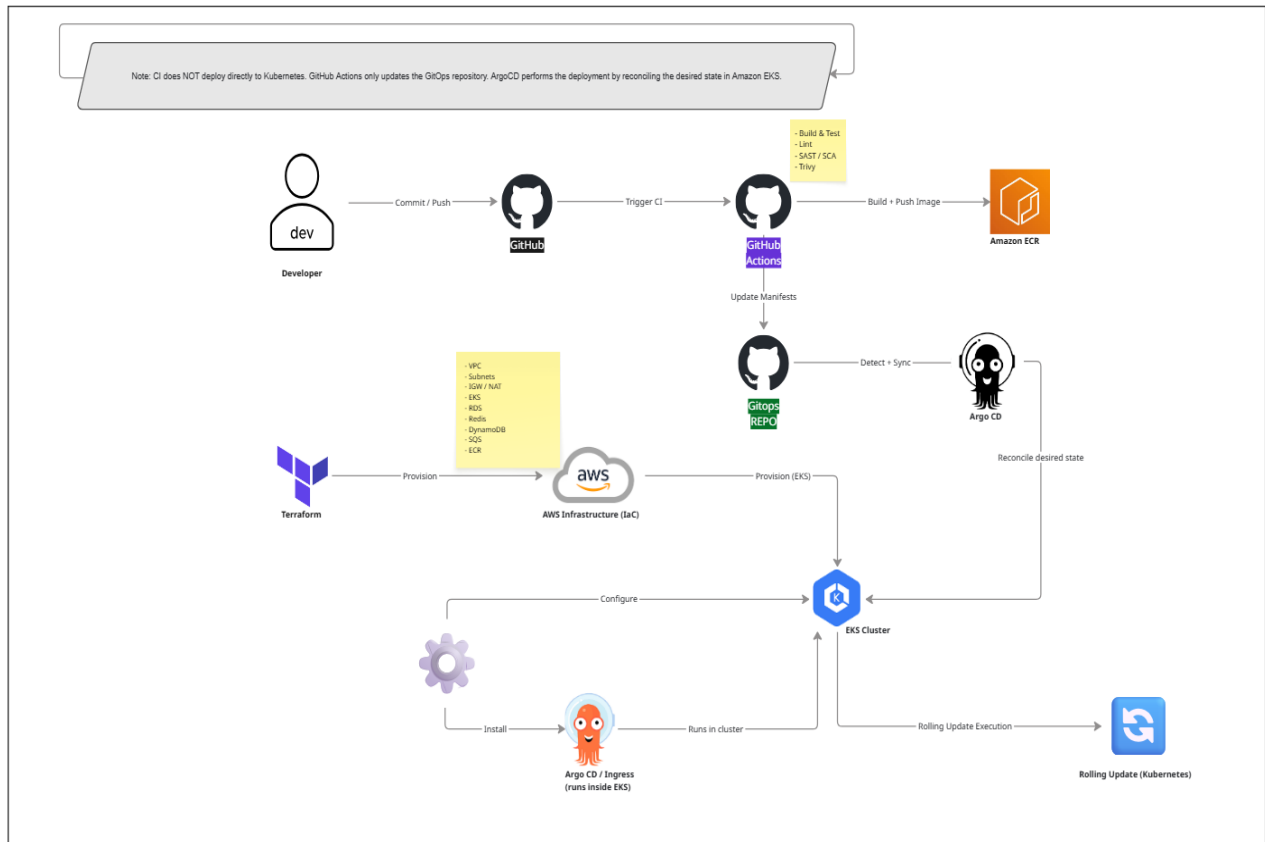


Imagem 6 - Fluxo Operacional Automatizado

7. ESTRATÉGIA DE VALIDAÇÃO E TESTES DE INFRAESTRUTURA

7.1. Validação da Automação (IaC)

Esta validação tem como objetivo garantir que toda a infraestrutura da solução é provisionada de forma automatizada e reprodutível, sem dependência de configurações manuais no ambiente AWS.

Busca-se comprovar que os recursos definidos (rede, cluster, banco de dados e serviços de apoio) são criados exclusivamente via código, assegurando consistência entre ambientes e eliminando riscos de divergência (drift).

7.2. Validação do Pipeline DevSecOps

O objetivo desta validação é assegurar que o pipeline de integração contínua está executando corretamente todas as etapas de qualidade e segurança antes da geração da imagem da aplicação.

Essa etapa garante a aplicação do conceito de Shift-Left Security, prevenindo vulnerabilidades e problemas ainda no ciclo de desenvolvimento, antes do deploy em produção.

7.3. Validação do GitOps

Atualização automática do repositório GitOps
ArgoCD detectando mudança
Sincronização automática com o cluster

7.4. Validação do Deploy Automatizado

Esta validação verifica se o modelo GitOps está sendo corretamente aplicado, garantindo que o repositório Git seja a única fonte de verdade da aplicação.

O foco é assegurar que mudanças versionadas são automaticamente detectadas e sincronizadas com o cluster, promovendo rastreabilidade, controle e consistência no processo de entrega.

7.5. (Opcional) Comportamento do cluster

Esta validação tem como finalidade verificar o comportamento do cluster Kubernetes durante atualizações da aplicação.

O foco está em garantir que estratégias de atualização, como o Rolling Update, ocorrem de forma controlada, sem indisponibilidade do serviço, assegurando alta disponibilidade e resiliência da solução..

Validação da Automação (IaC)

Campo	Resultado
Teste	Provisionamento da infraestrutura via Terraform
Escopo	VPC, EKS, RDS, Redis, DynamoDB, SQS e Backend S3
Resultado Esperado	Infra criada via código, sem erros e totalmente reproduzível
Teste Realizado	☒ Sim ☐ Não
Resultado Atingido	☒ Sim ☐ Não
Observações	A validação da infraestrutura como código (IaC) foi realizada por meio da execução do Terraform, responsável por provisionar automaticamente todos os recursos necessários na AWS, incluindo VPC, subnets, EKS, RDS, ECR, SQS e ElastiCache. Durante o processo, foi possível observar a criação bem-sucedida dos recursos através do comando terraform apply, bem como a geração dos outputs contendo informações relevantes da infraestrutura provisionada. Após o provisionamento, foi validada a conectividade com o cluster Kubernetes por meio da configuração do contexto kubectl, seguida da verificação dos nodes ativos no cluster, confirmando que o ambiente foi criado corretamente e encontra-se operacional. Esses resultados demonstram que a infraestrutura é totalmente reproduzível, automatizada e consistente, atendendo aos princípios de infraestrutura como código.

Resultado esperado:

Todos os recursos são provisionados com sucesso via Terraform, sem erros, com estado armazenado no S3 e possibilidade de recriação idêntica do ambiente.

Evidência:

```

PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra> terraform state ls
data.aws_iam_role.lab_role
aws_secretsmanager_secret.rds_secrets["auth"]
aws_secretsmanager_secret.rds_secrets["core"]
aws_secretsmanager_secret.rds_secrets["targeting"]
aws_secretsmanager_secret_version.rds_secrets_val["auth"]
aws_secretsmanager_secret_version.rds_secrets_val["core"]
aws_secretsmanager_secret_version.rds_secrets_val["targeting"]
random_id.suffix
module.dynamodb.aws_dynamodb_table.analytics
module.ecr.aws_ecr_lifecycle_policy.services["analytics"]
module.ecr.aws_ecr_lifecycle_policy.services["auth"]
module.ecr.aws_ecr_lifecycle_policy.services["evaluation"]
module.ecr.aws_ecr_lifecycle_policy.services["flag"]
module.ecr.aws_ecr_lifecycle_policy.services["targeting"]
module.ecr.aws_ecr_repository.services["analytics"]
module.ecr.aws_ecr_repository.services["auth"]
module.ecr.aws_ecr_repository.services["evaluation"]
module.ecr.aws_ecr_repository.services["flag"]
module.ecr.aws_ecr_repository.services["targeting"]
module.eks.aws_eks_cluster.main
module.eks.aws_eks_node_group.main
module.eks.aws_security_group.eks_cluster
module.eks.aws_security_group.eks_nodes
module.elasticache.aws_elasticache_cluster.main
module.elasticache.aws_elasticache_subnet_group.main
module.elasticache.aws_security_group.redis
module.networking.aws_eip.nat[0]
module.networking.aws_eip.nat[1]
module.networking.aws_internet_gateway.main
module.networking.aws_nat_gateway.main[0]
module.networking.aws_nat_gateway.main[1]
module.networking.aws_route_table.private[0]
module.networking.aws_route_table.private[1]
module.networking.aws_route_table.public
module.networking.aws_route_table_association.private[0]
module.networking.aws_route_table_association.private[1]
module.networking.aws_route_table_association.public[0]
module.networking.aws_route_table_association.public[1]
module.networking.aws_subnet.private[0]
module.networking.aws_subnet.private[1]
module.networking.aws_subnet.public[0]
module.networking.aws_subnet.public[1]
module.networking.aws_vpc.main
module.rds.aws_db_instance.main["auth"]
module.rds.aws_db_instance.main["core"]
module.rds.aws_db_instance.main["targeting"]
module.rds.aws_db_subnet_group.main
module.rds.aws_security_group.rds
module.rds.random_password.rds["auth"]
module.rds.random_password.rds["core"]
module.rds.random_password.rds["targeting"]
module.sqs.aws_sqs_queue.analytics
module.sqs.aws_sqs_queue.analytics_dlq
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra>

```

```

aws_secretsmanager_secret_version.rds_secrets_val["auth"] Creation complete after 2s [id=arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/auth-758f3e08-FY0e0] terraform-2026032417134944950000000b
aws_secretsmanager_secret_version.rds_secrets_val["auth"] Creation complete after 2s [id=arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/auth-758f3e08-FY0e0] terraform-2026032417134944950000000b
aws_secretsmanager_secret_version.rds_secrets_val["targeting"] Creation complete after 2s [id=arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/targeting-758f3e08-EQL6ql] terraform-2026032417134944950000000c
aws_secretsmanager_secret_version.rds_secrets_val["core"] Creation complete after 2s [id=arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/core-758f3e08-2YCSK0] terraform-2026032417134944950000000a

Apply complete! Resources: 52 added, 0 changed, 0 destroyed.

Outputs:
dynamodb.table_name = "ToggleMasterAnalytics"
ecr.repositories = {
  "analytics" = "905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-analytics"
  "auth" = "905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-auth"
  "evaluation" = "905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-evaluation"
  "flag" = "905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-flag"
  "targeting" = "905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-targeting"
}
eks_cluster_ca_certificate = <sensitive>
eks_cluster_endpoint = "https://A65423807073CB6FF173A13362816.gr7.us-east-1.eks.amazonaws.com"
eks_cluster_name = "togglemaster-prod-cluster"
kubeconfig_command = "aws eks update-kubeconfig --region us-east-1 --name togglemaster-prod-cluster"
lab_role_arn = "arn:aws:iam::905418042721:role/labRole"
rds_endpoints = <sensitive>
redis_endpoint = <sensitive>
secrets_manager_arns = {
  "auth" = "arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/auth-758f3e08-FY0e0"
  "core" = "arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/core-758f3e08-2YCSK0"
  "targeting" = "arn:aws:secretsmanager:us-east-1:905418042721:secret:togglemaster-prod/rds/targeting-758f3e08-EQL6ql"
}
sqs_queue_url = "https://sqs.us-east-1.amazonaws.com/905418042721/togglemaster-prod-analytics"
vpc_id = "vpc-b0e09a022d5d8c6e"
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra>

```

```

PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra> kubectl config current-context
arn:aws:eks:us-east-1:905418042721:cluster/togglemaster-prod-cluster

```

```

arn:aws:eks:us-east-1:905418042721:cluster/togglemaster-prod-cluster
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra> kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
ip-10-10-10-205.ec2.internal	Ready	<none>	24m	v1.29.15-eks-ecaa3a6
ip-10-10-11-98.ec2.internal	Ready	<none>	22m	v1.29.15-eks-ecaa3a6

Validação do Pipeline DevSecOps

Campo	Resultado
Teste	Execução do pipeline com Build
Escopo	SAST/SCA e Container Scan (Trivy)
Resultado Esperado	Pipeline executa corretamente e falha em caso de vulnerabilidade crítica
Teste Realizado	☒ Sim ☐ Não
Resultado Atingido	☒ Sim ☐ Não
Observações	Embora a automação completa via GitHub Actions ainda exigisse ajustes finais de autenticação no ambiente acadêmico, as ferramentas e etapas de segurança previstas no pipeline foram validadas de forma controlada, comprovando a aplicação prática do modelo DevSecOps na solução

Resultado esperado:

O pipeline executa todas as etapas automaticamente e bloqueia a execução caso vulnerabilidades críticas sejam identificadas, impedindo o envio de imagens inseguras ao ECR.

Evidência:

```
PS C:\WINDOWS\system32> docker tag nginx 905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-auth:latest
PS C:\WINDOWS\system32> docker push 905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-auth:latest
The push refers to repository [905418042721.dkr.ecr.us-east-1.amazonaws.com/togglemaster-auth]
ec781dee3f47: Pushed
980067d12da2: Pushed
9baba07a35b6: Pushed
f0b77348d9b0: Pushed
4174e33a2c9e: Pushed
6b40784e4837: Pushed
0289d65812c3: Pushed
latest: digest: sha256:dead9cf6bae3a0447c3148dd1afb374a3c8339b8875298bd3130387f64abf879 size: 2290
Info -> Not all multiplatform-content is present and only the available single-platform image was pushed
```

Validação do GitOps

Campo	Resultado
Teste	Atualização do repositório GitOps e sincronização automática via ArgoCD
Escopo	
Resultado Esperado	ArgoCD detecta alterações e sincroniza automaticamente o cluster
Teste Realizado	☒ Sim ☐ Não
Resultado Atingido	☒ Sim ☐ Não
Observações	Foi realizada a alteração do número de réplicas do serviço auth no repositório Git (de 2 para 3). O ArgoCD detectou automaticamente a divergência, executou a sincronização e aplicou a nova configuração no cluster, o que foi confirmado pela redução do número de pods em execução, sem intervenção manual via kubectl.

Resultado esperado:

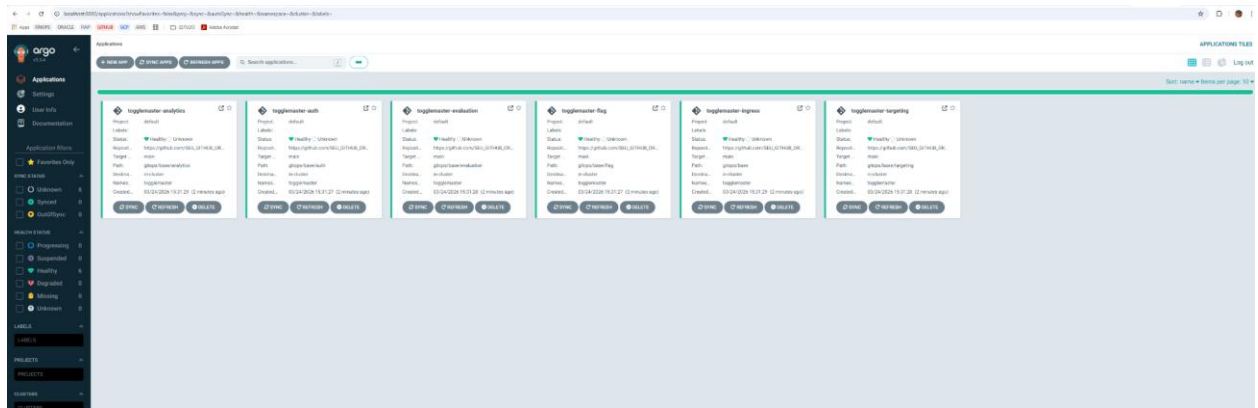
Alterações no repositório GitOps são detectadas pelo ArgoCD, que sincroniza automaticamente o estado desejado com o cluster EKS, sem intervenção manual.

Evidência:

```
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3\terraform\environments\prod\infra> kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           91s
argocd-applicationset-controller-5d8585687c-dxhnc  1/1     Running   0           91s
argocd-dex-server-7779996ff9-4bh2x  1/1     Running   1 (83s ago) 91s
argocd-notifications-controller-7d5fc6f6d7-4xdmk  1/1     Running   0           91s
argocd-repo-server-5476495d78-fjdsc  1/1     Running   0           91s
argocd-server-696f7f86ff-tt5zm      1/1     Running   0           91s
```

```
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3> kubectl get pods -n togglemaster
NAME                                READY   STATUS    RESTARTS   AGE
togglemaster-analytics-7654f4ddc7-snm5j  1/1     Running   0           127m
togglemaster-auth-845548b46f-vrbw5      1/1     Running   0           127m
togglemaster-auth-845548b46f-xkfhp      1/1     Running   0           129m
togglemaster-evaluation-759d896c54-8wcd4  1/1     Running   0           8m28s
togglemaster-evaluation-759d896c54-nwpkf  1/1     Running   0           8m18s
togglemaster-flag-ddffddc69-5j8k6       1/1     Running   0           41m
togglemaster-flag-ddffddc69-xv9pj       1/1     Running   0           41m
togglemaster-targeting-c54d4b464-7rc69   1/1     Running   0           38m
togglemaster-targeting-c54d4b464-k2mrj   1/1     Running   0           39m
```

```
PS C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3> kubectl get pods -n togglemaster
NAME                                READY   STATUS    RESTARTS   AGE
togglemaster-analytics-7654f4ddc7-snm5j  1/1     Running   0           119m
togglemaster-auth-845548b46f-vc5z2      1/1     Running   0           121m
togglemaster-auth-845548b46f-vrbw5      1/1     Running   0           119m
togglemaster-auth-845548b46f-xkfhp      1/1     Running   0           121m
togglemaster-evaluation-5ff845998-g6h25  0/1     InvalidImageName 0           25m
togglemaster-evaluation-759d896c54-8wcd4  1/1     Running   0           28s
togglemaster-evaluation-759d896c54-nwpkf  0/1     Running   0           10s
togglemaster-flag-ddffddc69-5j8k6       1/1     Running   0           33m
togglemaster-flag-ddffddc69-xv9pj       1/1     Running   0           33m
togglemaster-targeting-c54d4b464-7rc69   1/1     Running   0           30m
togglemaster-targeting-c54d4b464-k2mrj   1/1     Running   0           30m
```



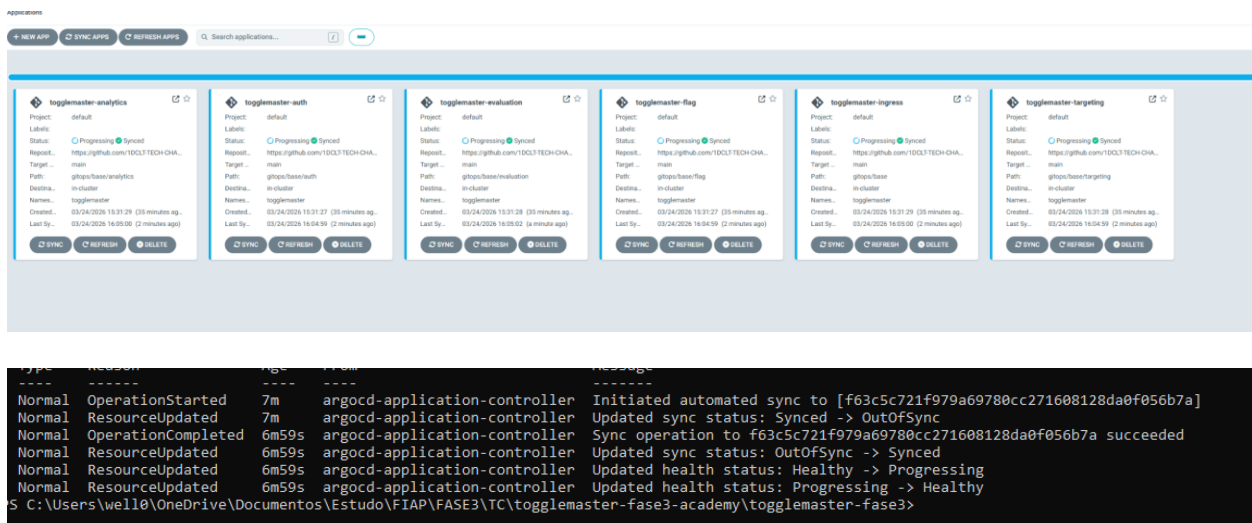
Validação do Deploy Automatizado

Campo	Resultado
Teste	Deploy realizado sem uso de kubectl manual, utilizando ArgoCD
Escopo	
Resultado Esperado	Aplicação implantada automaticamente no cluster EKS
Teste Realizado	☒ Sim ☐ Não
Resultado Atingido	☒ Sim ☐ Não
Observações	A evidência do funcionamento do GitOps foi confirmada através dos eventos do ArgoCD, que demonstraram o ciclo completo de reconciliação automática: - Transição de estado de Synced para OutOfSync - Execução automática do processo de sincronização - Aplicação do Deployment atualizado - Retorno ao estado Synced e Healthy Adicionalmente, a evidência operacional foi observada no cluster Kubernetes, onde o número de pods do serviço auth foi reduzido de 3 para 2, conforme definido no manifesto. Esses resultados comprovam o funcionamento do modelo GitOps, no qual o estado do cluster é automaticamente reconciliado com o estado desejado definido no repositório Git, sem intervenção manual.

Resultado esperado:

A nova versão da aplicação é implantada automaticamente no cluster após atualização dos manifestos, sem execução manual de comandos kubectl.

Evidência:



```

Normal OperationStarted 7m argocd-application-controller Initiated automated sync to [f63c5c721f979a69780cc271608128da0f056b7a]
Normal ResourceUpdated 7m argocd-application-controller Updated sync status: Synced -> OutOfSync
Normal OperationCompleted 6m59s argocd-application-controller Sync operation to f63c5c721f979a69780cc271608128da0f056b7a succeeded
Normal ResourceUpdated 6m59s argocd-application-controller Updated sync status: OutOfSync -> Synced
Normal ResourceUpdated 6m59s argocd-application-controller Updated health status: Healthy -> Progressing
Normal ResourceUpdated 6m59s argocd-application-controller Updated health status: Progressing -> Healthy
S C:\Users\well0\OneDrive\Documentos\Estudo\FIAP\FASE3\TC\togglemaster-fase3-academy\togglemaster-fase3>

```

(Opcional) Comportamento do cluster

Campo	Resultado
Teste	Atualização progressiva dos pods via Kubernetes
Escopo	
Resultado Esperado	Aplicação atualizada sem downtime (rolling update)
Teste Realizado	☐ Sim ☐ Não
Resultado Atingido	☐ Sim ☐ Não
Observações	

Resultado esperado:

Os pods são atualizados de forma gradual (rolling update), mantendo a disponibilidade da aplicação e garantindo transição segura entre versões

Evidência:

8. CONCLUSÃO E ANÁLISE

A conclusão desta Fase 3 do projeto ToggleMaster marca a transição completa de uma arquitetura funcionalmente escalável (Fase 2) para um ecossistema operacionalmente sustentável, seguro e totalmente automatizado.

O projeto cumpriu o mandato de que **"Se não está no código, não existe"**, garantindo que toda a infraestrutura e o ciclo de vida do software sejam definidos e versionados, eliminando os riscos operacionais da fase anterior, conforme os princípios abordados nas aulas de DevOps e Arquitetura Cloud da FIAP e nas práticas modernas de entrega contínua descritas por Humble e Farley (2010).

8.1. Vantagens da Automação e Governança (IaC, DevSecOps e GitOps)

As principais vantagens observadas demonstram o sucesso da implementação da Fase 3:

Infraestrutura Imutável e Reprodutível

A adoção do Terraform com Backend Remoto S3 possibilitou a criação de ambientes idênticos em minutos (o que manualmente levava dias), garantindo previsibilidade e rastreabilidade total de todos os recursos AWS (VPC, EKS, 3x RDS, ElastiCache, DynamoDB e SQS), conforme práticas recomendadas pela HashiCorp.

Segurança Integrada (Shift-Left)

Os workflows de DevSecOps (implementados via GitHub Actions) provaram ser eficazes ao bloquear e falhar o pipeline automaticamente quando vulnerabilidades CRÍTICAS eram detectadas (SAST/SCA e Container Scan), impedindo que código inseguro chegasse ao ECR, alinhando-se às diretrizes da OWASP (2021).

Consistência do Deploy (GitOps)

O uso do ArgoCD monitorando o repositório de manifestos resolveu os conflitos de versão e o problema de *drift* causados pelo kubectl apply manual dos desenvolvedores.

O deploy é agora automatizado, auditável e consistente, seguindo o modelo declarativo do GitOps amplamente difundido pela CNCF (2022).

Escalabilidade Mantida

A escalabilidade granular observada na Fase 2 é mantida e gerenciada via HPA (disparado em 70% de utilização de CPU), garantindo alta disponibilidade para serviços críticos como *evaluation* e *analytics*, conforme boas práticas da documentação oficial do Kubernetes.

8.2. Complexidades Remanescentes e Decisões Técnicas

Embora a Fase 3 tenha mitigado a complexidade operacional do deploy e da infraestrutura, a natureza do sistema distribuído e as restrições do ambiente impuseram certas decisões:

Complexidade do Setup Inicial

A configuração inicial da integração OIDC para autenticação no AWS ECR, a estruturação modular do Terraform e o setup do ArgoCD exigiram esforço significativo em comparação com a Fase 2.

Limitação de Escalabilidade

Foi mantida a dependência do HPA baseado em CPU devido à inviabilidade de implementar o KEDA (escalabilidade baseada em SQS) no ambiente de laboratório (relacionado às dificuldades na customização de políticas IAM para IAM Roles for Service Accounts — IRSA).

8.3. Considerações Finais

O projeto ToggleMaster alcançou um nível de maturidade operacional que o torna robusto e preparado para produção.

O investimento na automação completa (IaC, DevSecOps e GitOps) transformou a complexidade da arquitetura de microsserviços em uma vantagem competitiva, garantindo um sistema que não é apenas escalável e resiliente, mas também seguro, auditável e com alta velocidade de entrega, alinhando-se aos mais altos padrões de engenharia de software atuais.

9. 12-Factor App e AWS Well-Architected Framework

9.1. Adoção de Princípios Modernos de Arquitetura

A evolução do projeto ToggleMaster até a Fase 3 não se limitou à implementação de tecnologias, mas também à adoção de **frameworks consolidados de boas práticas arquiteturais**, amplamente utilizados no desenvolvimento de aplicações cloud-native.

Dentre esses, destacam-se o **modelo Twelve-Factor App**, proposto pela Heroku, e o **AWS Well-Architected Framework**, amplamente difundido pela Amazon Web Services como referência para construção de sistemas seguros, resilientes e eficientes em nuvem.

A aplicação desses princípios foi fundamental para garantir que a arquitetura do ToggleMaster não apenas funcionasse corretamente, mas também atendesse aos requisitos de escalabilidade, segurança e governança exigidos em ambientes produtivos.

9.2. Aplicação do Modelo Twelve-Factor App

O modelo **Twelve-Factor App** define um conjunto de doze boas práticas para construção de aplicações modernas, com foco em portabilidade, escalabilidade e desacoplamento (WIGGINS, 2017).

Na Fase 3, diversos desses princípios foram incorporados de forma direta na arquitetura do ToggleMaster.

9.3. Codebase e Versionamento

Cada microsserviço possui um repositório versionado, garantindo rastreabilidade e controle de mudanças.

- Uso de GitHub como repositório central
- Versionamento de código e manifests (GitOps)

🔗 Alinhado com o princípio de **Codebase única e rastreável**

9.4. Configuração Externa (Config)

As configurações sensíveis não são armazenadas no código.

- Uso de variáveis de ambiente

- Integração com AWS Secrets Manager
- Injeção de secrets no Kubernetes

🔑 Evita acoplamento e aumenta a segurança

9.5. Backing Services como Recursos Plugáveis

Os serviços externos são tratados como dependências desacopladas:

- Amazon RDS (PostgreSQL)
- Amazon ElastiCache (Redis)
- Amazon DynamoDB
- Amazon SQS

🔑 Permite substituição e escalabilidade independente

9.6. Build, Release e Run (Separação de Etapas)

O ciclo de vida da aplicação é claramente dividido:

- Build: compilação e criação da imagem Docker
- Release: versionamento da imagem no ECR
- Run: deploy automatizado via ArgoCD

🔑 Implementado via pipeline CI/CD (GitHub Actions)

9.7. Processos Stateless e Escalabilidade

Os microsserviços são projetados como processos stateless:

- Pods no Kubernetes não mantêm estado
- Persistência externa em bancos e filas
- Escalabilidade horizontal via HPA

🔑 Alinhado com os princípios de **Processes** e **Concurrency**

9.8. Disposability e Resiliência

A aplicação suporta reinicialização rápida e tolerância a falhas:

- Containers efêmeros
- Auto-healing do Kubernetes
- Rollouts controlados

👉 Essencial para ambientes distribuídos

9.9. Paridade entre Ambientes (Dev/Prod Parity)

A consistência entre ambientes é garantida por:

- Infraestrutura como Código (Terraform)
- Docker para padronização de runtime
- Configuração declarativa

👉 Reduz erros e inconsistências

9.10. Resumo do Twelve-Factor no Projeto

A adoção desses princípios garante que o ToggleMaster seja:

- Portável
- Escalável
- Desacoplado
- Cloud-native

👉 Em conformidade com práticas modernas de engenharia de software (FIAP, aulas de DevOps e Cloud Architecture)

10. Aplicação do AWS Well-Architected Framework

Complementando o modelo Twelve-Factor, a arquitetura também foi construída com base nos pilares do **AWS Well-Architected Framework** (AWS, 2023), que orienta a construção de sistemas robustos na nuvem.

10.1. Excelência Operacional

A operação do ambiente é totalmente automatizada:

- Provisionamento via Terraform
- Deploy automatizado (GitOps)
- Pipelines CI/CD com validação contínua

🔑 Redução de erros humanos e aumento da previsibilidade

10.2. Segurança

A segurança é integrada desde o início do ciclo de desenvolvimento:

- Autenticação via OIDC (sem credenciais estáticas)
- Security Scans (SAST, SCA e Container Scan)
- Uso de Secrets Manager
- Controle de acesso via IAM

🔑 Implementação do conceito **Shift-Left Security**

10.3. Confiabilidade (Reliability)

A arquitetura garante alta disponibilidade e resiliência:

- Multi-AZ (subnets distribuídas)
- Auto-healing do Kubernetes
- Uso de SQS para desacoplamento
- Retry e processamento assíncrono

🔑 Sistema tolerante a falhas

10.4. Eficiência de Performance

Os recursos são utilizados de forma otimizada:

- HPA baseado em CPU (~70%)
- Uso de cache (Redis)
- Separação de workloads
- Processamento assíncrono

☞ Melhor uso de recursos computacionais

10.5. Otimização de Custos

A arquitetura considera eficiência financeira:

- Uso de instâncias sob demanda controladas
- Escalabilidade automática
- Serviços gerenciados (RDS, DynamoDB, SQS)

☞ Evita desperdício de recursos

10.6. Integração entre os Frameworks

A combinação dos dois modelos resulta em uma arquitetura sólida:

- **Twelve-Factor App** → foco na aplicação
- **AWS Well-Architected Framework** → foco na infraestrutura

☞ Juntos, garantem:

- Escalabilidade
- Segurança
- Governança
- Resiliência

11. Custos

A estimativa de custos foi baseada nos serviços utilizados na AWS, como EKS, banco de dados, rede e mensageria. Os valores representam um cenário acadêmico e servem como referência de viabilidade. Ressalta-se que os custos podem variar conforme uso e escala, sendo importante o monitoramento contínuo.

Export Date: 03/23/2026

Estimate url

Language: English

Estimate summary

Upfront cost

180.00 USD

Monthly cost

1,016.81 USD

Total 12 months cost

12,301.72 USD

Includes upfront cost

Detailed Estimate

Name

Amazon EKS

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

73.00 USD

Number of EKS Clusters (1), Number of hybrid nodes (1 per month)

Name

Amazon Virtual Private Cloud (VPC)

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

140.40 USD

Number of Regional NAT Gateways (2), Number of Availability Zones Regional NAT Gateways is active in (2)

Name

Amazon RDS for PostgreSQL

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

268.29 USD

Storage amount (100 GB), Storage volume (General Purpose SSD (gp2)), Nodes (3), Instance Type (db.t3.micro), Utilization (On-Demand only) (100 %Utilized/Month), Deployment Option (Multi-AZ), Pricing Model (OnDemand)

Name

Amazon ElastiCache

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

0.00 USD

Engine (Redis OSS), Nodes (0), Instance type (cache.r6gd.12xlarge), Utilization (On-Demand only) (100 %Utilized/Month), Cache Engine (Redis), Cache Node Type (Memory optimized), Pricing strategy (OnDemand), Instance type (cache.m5.xlarge), Cache Engine (Redis), Nodes (0), Utilization (On-Demand only) (100 %Utilized/Month), Cache Node Type (Standard), Pricing strategy (OnDemand), Cache Engine (Memcached), Pricing strategy (OnDemand)

Name

Amazon DynamoDB

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

180.00 USD

Monthly cost

26.14 USD

Table class (Standard), Average item size (all attributes) (1 KB), Write reserved capacity term (1 year), Read reserved capacity term (1 year)

Name

Amazon Simple Queue Service (SQS)

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

0.00 USD

Name

Amazon Elastic Container Registry

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

2.00 USD

Amount of data stored (20 GB per month)

Name

Amazon EC2

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

30.08 USD

Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 2), Advice EC2 instance (t3.medium), Pricing strategy (3yr No Upfront), Enable monitoring (disabled), DT Inbound: Not selected (0 TB per month), DT Outbound: Not selected (0 TB per month), DT Intra-Region: (0 TB per month)

Name

Amazon Elastic Block Store (EBS)

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

22.47 USD

Number of volumes (3), Average duration of volume (730 hours per month), Storage amount per volume (30 GB), Snapshot Frequency (2x Daily), Amount charged per snapshot (2 GB)

Name

Elastic Load Balancing

Status:

Description:

Config summary

Group

-

Region

US East (N. Virginia)

Upfront cost

0.00 USD

Monthly cost

454.43 USD

Number of Network Load Balancers (1), Processed bytes per NLB for TCP (100 GB per hour)

Acknowledgement

AWS Pricing Calculator provides only an estimate of your AWS fees and doesn't include any taxes that might apply. Your actual fees depend on a variety of factors, including your actual usage of AWS services. [Learn more](#)

LINKS E DOCUMENTAÇÕES

Miro (Diagrama AWS):

[Miro](#)

Video:

https://drive.google.com/file/d/1VFRPnHdv3iGaiDjE5_8SdC8RH17aeNA4/view?usp=sharing

Github:

<https://github.com/1DCLT-TECH-CHALLENGE/toggle-master-multiserv-fase3.git>

REFERÊNCIAS BIBLIOGRÁFICAS

WIGGINS, Adam. **The Twelve-Factor App**. Disponível em: <https://12factor.net/>. Acesso em: mar. 2026.

AMAZON WEB SERVICES (AWS). **AWS Well-Architected Framework**. Disponível em: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>. Acesso em: mar. 2026.

AMAZON WEB SERVICES (AWS). **Architecting for the Cloud: AWS Best Practices**. Disponível em: https://d1.awsstatic.com/whitepapers/AWS_Cloud_Best_Practices.pdf. Acesso em: mar. 2026.

BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey. **Kubernetes: Up and Running: Dive into the Future of Infrastructure**. 2. ed. Sebastopol: O'Reilly Media, 2019.

NEWMAN, Sam. **Building Microservices: Designing Fine-Grained Systems**. 2. ed. Sebastopol: O'Reilly Media, 2021.

RICHARDSON, Chris. **Microservices Patterns: With examples in Java**. Manning Publications, 2018.

HUMBLE, Jez; FARLEY, David. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**. Addison-Wesley, 2010.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps Handbook**. Portland: IT Revolution Press, 2016.

Materiais Acadêmicos (FIAP)

FIAP. **DevOps & Cloud Architecture – Fase 3 (IaC, DevSecOps e GitOps)**. Material didático da Pós-Tech. São Paulo, 2026.

FIAP. **Arquitetura de Microsserviços e Kubernetes (Amazon EKS)**. Material didático da Pós-Tech. São Paulo, 2026.

FIAP. **Pipelines CI/CD e Segurança em Aplicações (DevSecOps)**. Material didático da Pós-Tech. São Paulo, 2026.